Design Decisions & Requirements Coverage

Companion to ARCHITECTURE.md. Explains what was chosen, why, and how each requirement of the assignment is handled — in the prototype and in production.

1. Technology choices

Choice	What	Why
Agent framework	**deepagents** (LangChain) on LangGraph	Production-grade agent loop with planning, checkpointing and — decisively — runtime-enforced human-in-the-loop interrupts, which is exactly the mechanism the destructive-ops requirement needs. Graph state checkpointing also gives crash recovery and multi-turn memory for free.
LLM (prototype)	`kimi-k3:cloud` via **Ollama Cloud**	Strong reasoning + tool-calling model; Ollama is one of the providers explicitly allowed by the assignment. A thinking model helps on multi-step "why" questions. Any Ollama Cloud model is a one-line `.env` change.
LLM (production)	**Gemini 2.5 Pro** primary, **Flash** fallback/judge, on Vertex AI	Same cloud as the data (BigQuery), IAM instead of API keys, per-project quota management, regional data residency. Flash is ~10x cheaper — right-sizing the judge and fallback paths controls cost.
Warehouse	**BigQuery**	Mandated by the assignment; serverless, byte-billing enables hard per-query cost caps.
Prototype storage	**SQLite**	Zero-setup, ships in the repo, and hides behind the same storage interface Firestore would implement.
CLI	**rich**	Required CLI-based UX: markdown rendering, tables, confirmation prompts.

2. Requirements, one by one

R1 — Hybrid Intelligence (Golden Bucket)

Prototype: `golden/` holds Question → SQL → Report trios. The `search_golden_examples` tool retrieves the most similar trios (lexical overlap scoring) and the system prompt obliges the agent to consult it before writing SQL. The trios carry analyst conventions that the schema cannot teach: revenue definition, how to decompose "underspending" or "churn", report structure executives expect.

Production: trios live in GCS, embedded into Vertex AI Vector Search; retrieval is semantic top-k with a similarity threshold.

Updating over time: see the learning-loop diagram in ARCHITECTURE.md §4 — positive feedback (👍, saved/shared reports) produces candidate trios that a human analyst reviews before publication; the index is rebuilt nightly. Human curation prevents feedback loops from poisoning the knowledge base.

R2 — Safety & PII Masking

Three independent layers:

1. **SQL guard (before execution)** — `src/safety/pii.py` + `src/tools/bigquery.py`: queries referencing `email`, `street_address`, `postal_code`, `latitude`, `longitude`, `user_geom` are rejected with an instructive message; only single-statement `SELECT` is allowed, DML/DDL keywords are blocked. Raw PII therefore never leaves BigQuery at all.
2. **Output masker (after generation)** — the final answer is regex-scanned for emails/coordinates and masked; the CLI shows a warning when masking fired. Safety net in case a value arrives via an unexpected path.
3. **Scope guardrail (prompt + evals)** — the agent only handles analysis of this dataset; off-topic and instruction-override attempts are declined (adversarial cases `pii_attempt`, `off_topic` in the eval suite verify it).

Production adds: read-only service account without permission to the PII columns (BigQuery column-level ACLs — enforcement moves from app code to IAM) and a Cloud DLP scan on outputs.

R3 — High-Stakes Oversight (destructive ops)

Report deletion is a real tool (`delete_saved_reports`) wrapped in a **LangGraph interrupt**: when the model calls it, the graph pauses, the CLI renders exactly what will be deleted (ids, titles,

model's stated reason) and resumes only with the human's approve/reject decision. Properties:

- **Runtime-enforced** — no prompt injection can skip the gate; the tool body is unreachable without an approval decision recorded in the graph state.
- **UX-preserving** — the model first narrows down which reports match ("all reports mentioning Client X" → concrete ids), so the user confirms once, with full context, instead of being interrogated.
- **Owner-scoped** — the storage layer filters by username on every operation; the model physically cannot delete another user's reports.

R4 — Continuous Improvement

User level: the `remember_preference` tool stores durable preferences (SQLite, per user). Preferences are injected into the system prompt of every subsequent turn and session — Manager A gets tables, Manager B gets bullet points, without re-asking. `/prefs` shows the stored profile.

Preferences evolve, they don't just accumulate: when the user changes their mind ("no more charts, give me bullets"), the agent calls `forget_preference` for the outdated entry and `remember_preference` for the new one, so the profile stays small and contradiction-free. `forget_preference` deletes by exact text, resolves an unambiguous partial match, and refuses ambiguous ones listing the options. At larger scale the production design adds periodic profile consolidation (raw preference events distilled into a fixed-size profile, newer wins on conflict).

System level: three mechanisms —

1. Golden Bucket curation (R1) turns good interactions into retrievable expertise for all users. The capture step is implemented in the prototype: `/good [note]` packages the last exchange (question + executed SQL + final report) into a candidate trio in `golden/candidates/`, invisible to retrieval until a human reviews it and moves it into `golden/` — from the next question on, the agent applies the newly learned pattern.
2. Negative feedback becomes eval regression cases (R6).
3. Traces (R7) expose failure clusters (e.g. queries that needed 3 SQL attempts), which drive prompt/tool fixes; in production this analysis runs on the Langfuse → BigQuery export.

R5 — Resilience & Graceful Error Handling

Failure	Handling
SQL syntax/semantic error	BigQuery error text is returned to the model as a tool observation; it corrects and retries — hard cap of 3 attempts per question (prompt-

Failure	Handling
	enforced, trace-verified), then an honest "here is what I could not do" answer. Bounded retries keep costs flat.
Empty result	Distinct `EMPTY_RESULT` observation instructing the model to re-check filter values before retrying, and to report honestly if genuinely empty (never invent numbers — verified by the `empty_result_recovery` eval).
LLM API failure / 429	Invoke wrapper: retry on the primary model → switch to the fallback model. Because conversation state is checkpointed, the retry *resumes* mid-turn instead of restarting it.
Query cost runaway	`maximum_bytes_billed` cap on every query (default 2 GB) — BigQuery refuses anything larger; `recursion_limit` caps the agent loop.
Total failure	The CLI catches everything, prints a friendly message, and the conversation continues — the UI never crashes.

R6 — Quality Assurance

`evals/run_evals.py` runs the agent end-to-end on a golden question set covering every advertised capability plus adversarial cases (PII extraction, prompt override, nonexistent entities). Each answer is scored by:

1. **Deterministic checks** — PII regex scan, raw-error leak, non-emptiness.
2. **LLM-as-judge** — 1–5 score against a per-case intent rubric.

A case passes at score ≥ 4 with zero deterministic failures; the run exits non-zero otherwise, so it slots into CI as a release gate. *Verifying intent*: the judge rubric encodes what a correct answer must contain (numbers, drivers, trend statement), not surface form. *Evaluating UX*: session metrics (turn latency, clarification rate, self-correction rate) plus in-product feedback (👍/👎) in production; the eval suite grows with every real-world failure.

R7 — Observability

Every LLM call and tool call is traced (`src/observability.py`) to a JSONL file per session: timestamps, latency, token usage, tool inputs/outputs (truncated), errors. In-session: `/trace` replays the reasoning chain step by step — you can see the exact SQL, the error the model received, and what it did next; `/metrics` shows counters.

The production path is wired in too: add Langfuse keys to `.env` and the same runs stream to **Langfuse** as nested traces — every turn is a `chat-turn` trace carrying `user_id`, `session_id` and a `persona:*` tag, with each LLM generation and tool call as child observations (latency, tokens, cost). Eval runs ship there as well, tagged `eval` + case id, so pre-deploy quality runs and live traffic land in one place. The integration is optional and fail-safe: missing keys or Langfuse downtime never affect the chat — local JSONL tracing is always on.

Agent-level metrics to alert on in production (Langfuse → Cloud Monitoring):

- error rate per stage (guard rejections, SQL errors, LLM failures)
- self-correction rate (SQL attempts per question — quality early-warning)
- p50/p95 turn latency; tokens and $ per turn (cost inflation guard)
- golden-retrieval hit rate; judge score on sampled live traffic
- destructive-op volume and reject rate

The trace answers "what was the message correspondence and what went wrong" by construction: it is the full correspondence.

R8 — Agility (Persona Management)

Personas are YAML files (`personas/*.yaml`) holding tone/structure instructions. The system prompt is rebuilt **on every turn**, so editing a YAML (or adding a new one) takes effect on the next message — no restart, no redeploy. `/persona board` switches at runtime.

Non-developers don't even need to touch the files: personas are edited **by conversation**. "Make the executive reports more formal and end each with a Risks section" → the agent reads the current persona, proposes a revised style preserving what should be kept, and the change goes through the same confirmation interrupt as report deletion — the user sees a current-vs-proposed diff and approves or rejects. `update_persona` can also create new personas on request. Since a persona affects everyone who uses it, the human-approval gate is runtime-enforced, not prompt-enforced.

In production the same content lives in Firestore behind a small admin UI with versioning and instant propagation; the chat-editing path stays available — the CEO's weekly tone change is a chat message or a form edit, never a release.

3. Cost control summary

Bounded SQL retries (3), byte-capped queries (2 GB), recursion-limited agent loop (50), capped result rows into context (50), fallback to the same-or- cheaper model — every unbounded loop in the system has an explicit ceiling.